Comprehensive Implementation Framework for Deriving COLMAP-Compatible Sparse Models from Planet SuperDove L1A Imagery

1. Introduction

The integration of satellite imagery into advanced neural rendering pipelines, particularly 3D Gaussian Splatting (3DGS), represents a frontier in geospatial photogrammetry. While traditional Structure-from-Motion (SfM) workflows have matured for frame-based aerial and terrestrial photography, the utilization of raw satellite sensor data—specifically the Level 1A (L1A) "butcher block" imagery from Planet Labs' SuperDove constellation—introduces a unique set of geometric and radiometric challenges. This report articulates a rigorous implementation plan to derive a sparse model compatible with COLMAP (and subsequently 3DGS) from such data.

The project's core objective is to bypass the standard orthorectification pipeline, which introduces resampling artifacts detrimental to view synthesis, and instead model the physical acquisition geometry of the SuperDove sensor directly. By treating the L1A data as a sequence of raw sensor measurements, we preserve the radiometric integrity required for high-fidelity Gaussian Splatting.[1] This approach necessitates a sophisticated translation between the inertial reference frames of orbital mechanics and the Euclidean local tangent planes (LTP) governing computer vision algorithms.

The scope of this implementation encompasses the mathematical derivation of camera poses from ephemeris metadata, the interpretation of the proprietary "psblue" distortion model, and the generation of a sparse point cloud via ray-casting against a Digital Elevation Model (DEM). Crucially, the framework incorporates a validation mechanism utilizing Rational Polynomial Coefficients (RPCs) as a geometric ground truth to ensure that the derived pinhole approximations maintain sub-pixel accuracy, a requirement for preventing "floaty" artifacts in neural rendering.[3]

1.1 The SuperDove Optical Architecture

To accurately model the imaging process, one must first deconstruct the SuperDove (PSB.SD) instrument. Unlike traditional push-broom scanners that acquire images line-by-line, or monolithic frame sensors used in aerial photogrammetry, the SuperDove utilizes a 47-megapixel 2D CMOS array equipped with a "butcher block" filter assembly.[4] This filter divides the focal plane into eight discrete horizontal stripes, each corresponding to a specific

spectral band (Coastal Blue, Blue, Green I, Green, Yellow, Red, Red Edge, NIR).[5]

Table 1: SuperDove (PSB.SD) Sensor Characteristics

Characteristic	Specification	Implication for Implementation
Sensor Type	2D CMOS Frame	Enables frame-based SfM, unlike push-broom sensors.
Filter Layout	Butcher Block (8 Stripes)	Bands are spatially offset; they image the same ground target at different times ($t_0 +$).
Product Level	Level 1A (Basic Scene)	Unrectified, raw geometry. Pixels map to sensor elements, not map coordinates.[7]
Metadata	JSON (Ephemeris/Attitude)	Provides quaternion orientation and ECEF position.[8]
Distortion	Optical (Rad/Tan)	Significant distortion must be modeled via OpenCV parameters.[9]

The user's directive to treat all pixels as a "single color band" simplifies the temporal complexity. We assume the extraction of a single spectral stripe (e.g., the Green band) from the L1A stack. This reduces the problem to modeling a single frame camera with a unique principal point and distortion profile, captured at the timestamp associated with that specific stripe's integration center.[10]

---

2. Mathematical Framework: Coordinate Systems and Transformations

The fundamental challenge in this implementation is the dissonance between the coordinate systems used in astrodynamics (Planet Labs metadata) and those used in computer vision

(COLMAP/OpenCV). A precise, rigorous chain of transformations is required to bridge this gap.

2.1 Reference Frame Definitions

The implementation relies on four distinct coordinate frames. Understanding the orientation and origin of each is critical for avoiding the "coordinate system mismatch" errors mentioned in the project goals.

1. **The Sensor Frame (S):** This is the 2D pixel coordinate system of the raw L1A image. The origin $(0, 0)$ is the top-left pixel. The "psblue" camera model defines the mapping from 3D camera coordinates to this 2D plane.[10]
2. **The Spacecraft Body Frame (B):** A 3D Cartesian frame fixed to the satellite bus. The orientation of this frame relative to the Earth is given by the attitude quaternions in the metadata. Planet Labs documentation indicates that for the basic_l1a_all_frames asset, the attitude quaternion describes the rotation from the **ECEF frame to the Boresight frame**.[8] It is standard for the Boresight Z-axis to align with the optical axis.
3. **Earth-Centered, Earth-Fixed (E):** The global WGS84 coordinate system (X, Y, Z) in meters. The satellite's position is reported in this frame.[8]
4. **Local Tangent Plane (L):** A local Euclidean system (East-North-Up or ENU) centered at an arbitrary reference point P_{ref} within the scene. COLMAP and Gaussian Splatting utilize single-precision floating-point arithmetic in many stages, which leads to catastrophic precision loss when handling raw ECEF coordinates (magnitude $\sim 6 \times 10^6$ meters). Establishing an LTP is mandatory for numerical stability.[13]

2.2 The Transformation Chain

The goal is to derive the **World-to-Camera** transformation (T_{WC}) required by COLMAP, where "World" corresponds to our Local Tangent Plane (L). This transformation is composed of the satellite's extrinsic parameters.

Let P_L be a point in the Local Tangent Plane. The transformation sequence to the Camera Frame (C) is:

$$P_C = T_{B \to C} \cdot T_{E \to B} \cdot T_{L \to E} \cdot P_L$$

We dissect each component of this chain below.

2.2.1 Local to ECEF ($T_{L \to E}$)

The inverse of the ENU conversion. If the local origin is $O_L = (\phi_0, \lambda_0, h_0)$, the rotation matrix $R_{L \to E}$ is constructed from the latitude and longitude:

$$R_{L \to E} = \begin{bmatrix} -\sin\lambda & -\sin\phi\cos\lambda & \cos\phi\cos\lambda \\ \cos\lambda & -\sin\phi\sin\lambda & \cos\phi\sin\lambda \\ 0 & \cos\phi & \sin\phi \end{bmatrix}$$

The translation is simply the ECEF coordinate of the origin, $t_{L \to E} = O_{ECEF}$.

2.2.2 ECEF to Body ($T_{E \to B}$)

The Planet Labs metadata provides a quaternion $q_{sat} = [q_w, q_x, q_y, q_z]$. According to snippet [8], this quaternion describes the rotation **from ECEF to the Boresight frame**.

Therefore, the rotation matrix $R_{E \to B}$ can be computed directly from q_{sat} using standard Hamilton or JPL conventions (Planet documentation typically follows the standard engineering convention where q_{real} is the first or last element; verification via the scipy.spatial.transform library is recommended).

$$R_{E \to B} = \text{QuaternionToMatrix}(q_{sat})$$

The translation component involves the satellite's position C_{sat} in ECEF. The transformation of a point P_E into the body frame is:

$$P_B = R_{E \to B}(P_E - C_{sat})$$

2.2.3 Body to Camera ($T_{B \to C}$)

This is a static transformation representing how the sensor is mounted on the satellite bus.

- **Satellite Convention:** Typically, the Body frame aligns $+Z$ with the Boresight (pointing at Earth), $+X$ with the velocity vector, and $+Y$ with the cross-track direction.
- **COLMAP/OpenCV Convention:** The camera frame requires $+Z$ pointing forward

(optical axis), $+X$ pointing Right, and $+Y$ pointing Down.[3]

If the satellite Boresight $+Z$ points to the scene, and COLMAP $+Z$ also points to the scene, the axes might still be rotated about Z. However, a common discrepancy is the "Look" direction. If Boresight is defined as $+Z$, we need to ensure the image X/Y axes align. Usually, a fixed rotation R_{fix} is required. A common conversion from standard aerospace (Z-down) to computer vision (Z-forward) involves a 180-degree rotation around the X-axis.

Diagnostic Insight: To determine this matrix definitively without physical specs, the implementation plan includes a "Diagnostic Output" step (Section 5) to visualize projected axes on the ground.

2.2.4 Final COLMAP Pose (T_{WC})

COLMAP defines pose as R_{WC} (Rotation from World to Camera) and t_{WC} (Translation in Camera frame).

Combining the chains:

$$
\begin{aligned}
R_{WC} &= R_{fix} \cdot R_{E \to B} \cdot R_{L \to E} \\
C_{camera\_world} &= R_{L \to E}^{T} (C_{sat} - O_{ECEF}) \\
t_{WC} &= -R_{WC} \cdot C_{camera\_world}
\end{aligned}
$$

This derivation provides the exact QW, QX, QY, QZ and TX, TY, TZ values required for the images.txt file.

---

3. Deriving the OpenCV Camera Model

The prompt provides a specific dictionary of camera model parameters (psblue_camera_models). Correctly mapping these to the OpenCV model used by COLMAP is crucial for pixel-perfect alignment.

3.1 Interpreting 'rem' vs. 'add'

The provided code snippet contains two sets of distortion coefficients:

- 'rem': Likely stands for **Removal**. These coefficients define the function $f_{undistort} : P_{distorted} \to P_{ideal}$. This is used to rectify raw images.

- 'add': Likely stands for **Addition**. These coefficients define the function $f_{distort} : P_{ideal} \rightarrow P_{distorted}$.

Decision Matrix: Standard photogrammetric models (Brown-Conrady), including those in OpenCV and COLMAP, define distortion as a process that moves pixels *from* their ideal projection positions *to* their actual distorted positions on the sensor.[16]

$$x_{distorted} = x_{ideal}(1 + k_1 r^2 + k_2 r^4 + \dots)$$

Therefore, the implementation must use the **add** coefficients to parameterize the COLMAP model. Using the rem coefficients would invert the distortion correction, doubling the error rather than correcting it.

3.2 Parameter Scaling and Normalization

A critical observation from the provided snippet is the magnitude of the coefficients:

- $k_1 \approx -9.47 \times 10^{-10}$
- $k_2 \approx -2.67 \times 10^{-19}$

In the standard OpenCV model, r is the **normalized** radial distance ($r = \sqrt{x^2 + y^2}/z$), where $r \approx 1.0$ at the image edge. Coefficients for normalized coordinates are typically in the range of $[-1.0, 1.0]$. The extremely small values in the snippet indicate that the Planet Labs model applies to **pixel coordinates** (where r is measured in pixels, e.g., $r \approx 4000$).

To make these compatible with COLMAP's OPENCV model (which expects normalized coordinates), we must apply a scaling factor based on the focal length f (in pixels).

r_{pixel} = f \cdot r_{normalized}$$Substituting this into the distortion equation:$$\Delta x_{pixel} = x_{pixel} (k_{1,pix} r_{pixel}^2 + k_{2,pix} r_{pixel}^4)

\Delta x_{norm} \cdot f = (x_{norm} \cdot f) (k_{1,pix} (f \cdot r_{norm})^2 + k_{2,pix} (f \cdot r_{norm})^4)$$Dividing by f:$$\Delta x_{norm} = x_{norm} ([k_{1,pix} f^2] r_{norm}^2 + [k_{2,pix} f^4] r_{norm}^4)

Conversion Formula:

- $k_{1,colmap} = k_{1,planet} \cdot f^2$
- $k_{2,colmap} = k_{2,planet} \cdot f^4$

The focal length f must be extracted from the metadata or derived from the Field of View (FOV) and sensor width. PlanetScope sensors typically have a GSD of ~3m at 475km altitude, which allows for back-calculation of f if not explicitly stated in the JSON.

3.3 Principal Point Configuration

The snippet provides center offsets:

'center': {'x': CAMERA_CENTER_X + 53.478401, 'y': CAMERA_CENTER_Y - 21.147406}

COLMAP defines the principal point (c_x, c_y) in absolute pixel coordinates.

$$c_x = (W/2) + 53.478401$$

$$c_y = (H/2) - 21.147406$$

These offsets are significant and confirm the "butcher block" nature where the optical axis is not centered on the specific band's stripe.

---

4. Implementation Plan: The Processing Pipeline

This section details the Python-based pipeline to generate the sparse model.

4.1 Phase 1: Metadata Parsing and Trajectory Extraction

The first module, parse_and_convert.py, ingests the L1A JSON and GeoTiff.

1. **Data Loading:** Utilize json to load the metadata and tifffile or rasterio to read image dimensions (W, H).
2. **Ephemeris Extraction:** Retrieve the spacecraft_position (ECEF vectors) and attitude_quaternion.
3. **LTP Initialization:** Calculate the centroid of the image footprints. This point becomes O_{LTP}. Using pyproj or custom linear algebra, compute the orthogonal rotation matrix $R_{L \to E}$.
4. **Pose Computation:** Apply the transformation chain defined in Section 2.2 to generate q_{colmap} and t_{colmap} for every frame.
    - *Correction Check:* Ensure quaternions are normalized. COLMAP requires the

Hamilton convention (w, x, y, z). If the input is (x, y, z, w), permute indices.

4.2 Phase 2: Ray-Casting for Sparse Point Generation

COLMAP requires an initial set of 3D points to define the scene structure, especially when camera poses are fixed (known). We generate these by "back-projecting" pixels onto the Earth's surface using a DEM. This creates a "perfect" sparse cloud that aligns with the satellite's navigation data.

Algorithm: Iterative Ray-DEM Intersection

1. **Grid Sampling:** Select a sparse grid of pixels p_{ij} (e.g., stride of 50 pixels) across the image.
2. **Ray Generation (Camera Frame):** For each pixel, compute the normalized vector v_{cam} using the inverse of the intrinsic matrix K.

$$v_{cam} = \text{undistort}([u, v]) \rightarrow [x_n, y_n, 1]^T$$

3. **Ray Transformation (ECEF Frame):** Transform the ray into ECEF.

$$v_{ecef} = R_{E \rightarrow B}^T \cdot R_{B \rightarrow C}^T \cdot v_{cam}$$

   The ray origin is the satellite position C_{sat}.
4. **Intersection Search:**
   - Define a ray $R(t) = C_{sat} + t \cdot v_{ecef}$.
   - **Coarse Search:** March along t starting from altitude ≈ 500 km down to 0 km. At each step, convert $R(t)$ to Geodetic coordinates (ϕ, λ, h). Compare ray altitude h with DEM elevation $H_{dem}(\phi, \lambda)$ sampled via rasterio. Stop when $h < H_{dem}$.
   - **Fine Refinement:** Perform a binary search or secant method between the last two steps to find the precise intersection t^* where $h(t^*) = H_{dem}$.
5. **Output:** Convert the intersection point $P_{ecef}(t^*)$ to the Local Tangent Plane P_L and store it.

4.3 Phase 3: File Formatting for COLMAP

The data must be serialized into COLMAP's specific text format.[15]

- **cameras.txt:**
  - Structure: CAMERA_ID MODEL WIDTH HEIGHT params...

    - Content: 1 OPENCV W H fx fy cx cy k1 k2 0 0
- **images.txt:**
    - Structure: IMAGE_ID QW QX QY QZ TX TY TZ CAMERA_ID NAME
    - Followed by an empty line or dummy 2D points (since we are providing 3D points directly, we can leave points2D empty or link them if creating a full bundle adjustment input).
- **points3D.txt:**
    - Structure: POINT3D_ID X Y Z R G B ERROR TRACK
    - Populate with the XYZ coordinates from Phase 2. RGB values can be sampled from the image at the generating pixel coordinates.

---

5. Verification Strategy: The RPC Check

The prompt requires a verification step utilizing Estimated RPCs. This acts as a crucial "sanity check" to ensure the derived Pinhole + Distortion model accurately approximates the rigorous physical sensor model.[3]

The Verification Algorithm:

1. **Select Test Set:** Choose a random subset of 100 3D points $\{P_L\}$ generated in Phase 2.
2. **Forward Projection (COLMAP):** Project P_L using the derived COLMAP intrinsics ($K, dist$) and extrinsics (R, t).

$$p_{col} = \text{Project}_{pinhole}(P_L)$$

3. **Forward Projection (RPC):** Convert P_L to Geodetic coordinates (ϕ, λ, h). Use the RPC coefficients (from metadata) to compute the row/column (r, c).

$$p_{rpc} = \text{RPC}(\phi, \lambda, h)$$

4. **Residual Computation:** Calculate the Euclidean distance $\delta = ||p_{col} - p_{rpc}||$.
5. **Thresholding:**
    - If mean $\delta < 1.0$ pixel: **Success.** The geometric pipeline is consistent.
    - If mean $\delta > 5.0$ pixels: **Failure.** Indicates a coordinate system misalignment (e.g., axes flipped), incorrect focal length estimation, or errors in distortion scaling.

Rationale: RPCs are derived by the satellite provider (Planet) from the rigorous model. If our derived Pinhole model matches the RPCs, it means our interpretation of the physical metadata

(q_{sat}, C_{sat}) and our conversion logic are correct.

---

6. Diagnostic Output and Error Identification

To fulfill the requirement for "rapid identification of potential errors," the pipeline will generate a suite of diagnostic visualizations.

6.1 Diagnostic 1: The Residual Heatmap

Purpose: Detect systematic distortion errors or rotational biases.

- **Method:** Compute the RPC vs. Pinhole residual for a dense grid of points across the image.
- **Visualization:** A 2D heatmap overlaid on the image domain.
- **Interpretation:**
  - *Uniform color:* Good alignment.
  - *Radial gradient:* Incorrect k_1, k_2 scaling (Intrinsics error).
  - *Linear gradient:* Boresight misalignment or timing offset (Extrinsics error).

6.2 Diagnostic 2: World Axes Projection

Purpose: Verify the rotation matrix convention ($R_{B \to C}$).

- **Method:** Project unit vectors representing East (1,0,0), North (0,1,0), and Up (0,0,1) from the LTP origin into the camera view.
- **Visualization:** Arrows drawn on the image center.
- **Interpretation:**
  - The "Up" arrow should point towards the camera center (or vanish if strictly nadir).
  - The "North" arrow should align with the actual cardinal direction of the image (checked against satellite_heading in metadata). If the image is descending (sun-synchronous), North should generally be "up" or "down" depending on the flight path orientation.

6.3 Diagnostic 3: Frustum Visualization

Purpose: Detect gross positioning errors (e.g., altitude scaling).

- **Method:** Export a .ply file containing the camera frustums and the bounds of the generated point cloud.
- **Visualization:** Viewable in MeshLab.
- **Interpretation:** Ensure cameras are positioned ~475km *above* the point cloud, not inside it or below it.

7. Preparation for Gaussian Splatting

The final goal is to use this data for 3D Gaussian Splatting. 3DGS implementations are sensitive to scale and initialization.

7.1 Scale Normalization

ECEF coordinates are massive (10^6), and even ENU coordinates can be large (10^4). 3DGS often assumes the scene fits within a unit cube $[-1, 1]$ or similar small bounds.

- **Recommendation:** Apply a global scaling factor S to the points3D.txt and t vector in images.txt. A scale factor of $1/1000$ (converting meters to kilometers) or normalizing by the scene radius is recommended. This prevents numerical instability during the Gaussian densification process.[19]

7.2 Densification Strategy

Standard SfM often produces sparse clouds. Our ray-casting approach generates a "semi-dense" cloud (limited only by DEM resolution). This provides an excellent initialization for 3DGS, potentially allowing for faster convergence and fewer "floater" artifacts compared to initialization from random points.

7.3 Band Alignment Warning

While the user assumes a "single color band," if future steps integrate RGB, the "butcher block" parallax must be handled. 3DGS assumes a consistent center of projection for all channels. Using raw RGB L1A data without separating the bands into distinct "cameras" (each with its own slightly offset pose) will result in chromatic aberration artifacts in the reconstruction. The proposed pipeline treats the dataset as monochromatic to enforce geometric rigor.

8. Conclusion

This research report establishes a comprehensive, mathematically rigorous framework for ingesting Planet SuperDove L1A imagery into COLMAP and Gaussian Splatting pipelines. By deriving camera poses directly from inertial telemetry and validating them against provider-generated RPCs, we bypass the degradation associated with traditional orthorectification. The inclusion of the "psblue" distortion model, properly scaled from pixel to normalized coordinates, ensures that the lens characteristics are accurately represented.

Finally, the ray-casting strategy leverages existing DEM data to provide a robust geometric foundation, ensuring the successful initialization of neural rendering algorithms. This workflow effectively unlocks the high-cadence monitoring capabilities of the SuperDove constellation for advanced 3D reconstruction tasks.

Table 2: Implementation Checklist and Files

Component	File Name	Function
Parser	parse_metadata.py	Extracts q_{sat}, C_{sat}, focal length, and scales distortion k.
Geometry	coord_transform.py	Implements ECEF $\rightarrow$ ENU $\rightarrow$ Camera chain.
Model Gen	ray_caster.py	Intersects camera rays with DEM to create points3D.txt.
Validation	rpc_check.py	Computes residuals between Pinhole and RPC projections.
Export	colmap_exporter.py	Writes formatted images.txt, cameras.txt.
Diagnostics	plot_errors.py	Generates heatmaps and axes visualizations.

Works cited

1. Analysis-Ready PlanetScope - Planet Documentation, accessed February 9, 2026, https://docs.planet.com/data/imagery/arps/
2. Sat-DN: Implicit Surface Reconstruction from Multi-View Satellite Images with Depth and Normal Supervision - arXiv, accessed February 9, 2026, https://arxiv.org/html/2502.08352v1
3. Leveraging Vision Reconstruction Pipelines for Satellite Imagery - GitHub Pages, accessed February 9, 2026, https://kai-46.github.io/VisSat/my_files/paper.pdf

4. PlanetScope - Earth Online, accessed February 9, 2026, https://earth.esa.int/eogateway/missions/planetscope
5. PlanetScope | Planet Documentation, accessed February 9, 2026, https://docs.planet.com/data/imagery/planetscope/
6. Evaluation of Multi-Spectral Band Efficacy for Mapping Wildland Fire Burn Severity from PlanetScope Imagery - MDPI, accessed February 9, 2026, https://www.mdpi.com/2072-4292/15/21/5196
7. Technical Specification | Planet Documentation, accessed February 9, 2026, https://docs.planet.com/data/imagery/arps/techspec/
8. BASIC L1A All-Frames - USER GUIDE, accessed February 9, 2026, https://assets.planet.com/docs/Planet_Basic_L1A_All-Frames_User_Guide.pdf
9. On-Ground Calibration and Optical Alignment for the Orion Optical Navigation Camera, accessed February 9, 2026, https://ntrs.nasa.gov/api/citations/20180004169/downloads/20180004169.pdf
10. Combined Imagery Product Spec FINAL | October 2025 - Planet Labs, accessed February 9, 2026, https://assets.planet.com/docs/Planet_Combined_Imagery_Product_Specs_letter_screen.pdf
11. PLANET IMAGERY PRODUCT SPECIFICATIONS - ESA Earth Online, accessed February 9, 2026, https://earth.esa.int/eogateway/documents/20142/37627/Planet-combined-imagery-product-specs-2020.pdf
12. Transformations between ECEF and ENU coordinates - Navipedia - GSSC, accessed February 9, 2026, https://gssc.esa.int/navipedia/index.php/Transformations_between_ECEF_and_ENU_coordinates
13. ecef2enu - Transform geocentric Earth-centered Earth-fixed coordinates to local east-north-up - MATLAB - MathWorks, accessed February 9, 2026, https://www.mathworks.com/help/map/ref/ecef2enu.html
14. converting pose (which is a quaternion & a vector) from a coordinate system to another - Math Stack Exchange, accessed February 9, 2026, https://math.stackexchange.com/questions/4868220/converting-pose-which-is-a-quaternion-a-vector-from-a-coordinate-system-to-a
15. Output Format — COLMAP 3.14.0.dev0 | 5b9a079a (2025-11-14) documentation, accessed February 9, 2026, https://colmap.github.io/format.html
16. Camera Calibration - Duke Computer Science, accessed February 9, 2026, https://courses.cs.duke.edu/cps274/fall16/notes/calibration.pdf
17. VPI - Vision Programming Interface: Lens Distortion Correction - NVIDIA Documentation, accessed February 9, 2026, https://docs.nvidia.com/vpi/algo_ldc.html
18. Camera Calibration and 3D Reconstruction - OpenCV Documentation, accessed February 9, 2026, https://docs.opencv.org/3.4/d9/d0c/group__calib3d.html
19. Help to use ground-truth poses with COLMAP · Issue #706 · graphdeco-inria/gaussian-splatting - GitHub, accessed February 9, 2026, https://github.com/graphdeco-inria/gaussian-splatting/issues/706